



Software-Architekturen

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Informatik und
Wirtschaftsinformatik**

Die Vorlesung basiert im Wesentlichen auf Literatur und Materialien aus folgenden Quellen.

- Gorton, Ian:
Essential Software Architecture, 2. ed. Springer, 2011.
- Clements, P.; Bass, L. and Kazman, R.:
Software Architecture in Practice, 2. ed. Addison-Wesley, 2003.
- Vogel, O.; et al.:
Software-Architektur: Grundlagen – Konzepte – Praxis, 2. Auflage. Spektrum, 2009.
- Avgeriou, P.; et al. (editors):
Relating Software Requirements and Architectures. Springer, 2011.
- Larman, Craig:
Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3. ed. Prentice Hall, 2004.
- Sommerville, Ian:
Software Engineering, 9. Auflage. Pearson Studium, 2012.
- Fowler, M.:
Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.
- Richardson, C.:
Microservices Patterns: With Examples in Java. Manning, 2019.

- Heineman, G. T. and Councill, W. T.:
Component-Based Software Engineering: Putting the Pieces Together. Addison-Wesley, 2001.
- Gamma, Erich et al.:
Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1995.
- Fowler, Martin: Analysemuster:
Wiederverwendbare Objektmodelle. Addison-Wesley, 1999.
- Pattern-Oriented Software Architecture, Volume 1:
Buschmann, Frank:
A System of Patterns. John Wiley & Sons, 1996.
- Pattern-Oriented Software Architecture, Volume 2:
Schmidt, Douglas C.:
Patterns for Concurrent and Networked Objects. John Wiley & Sons, 2000.
- Pattern-Oriented Software Architecture, Volume 3:
Kircher, M. and Prashant, J.:
Patterns for Resource Management. John Wiley & Sons, 2004.
- Pattern-Oriented Software Architecture, Volume 4:
Buschmann, Frank; Henney, Kevlin; and Schmidt, Douglas C.:
A Pattern Language for Distributed Computing. John Wiley & Sons, 2007.
- Vernon, V.:
Domain-Driven Design kompakt. dpunkt.verlag, 2017.

- Reussner, R. und Hasselbring, W. (Hrsg.):
Handbuch der Software-Architektur. dpunkt, 2006.
- Goll, J. und Dausmann, M.:
Architektur- und Entwurfsmuster der Softwaretechnik. Springer Vieweg, 2013.
- Horn, E. und Reinke, Th.:
Softwarearchitektur und Softwarebauelemente. Hanser, 2002.
- Andresen, A.:
Komponentenbasierte Softwareentwicklung mit MDA und UML 2 und XML, 2. Auflage. Hanser, 2004.
- Rupp, C., Queins, S. und Zengler, B.:
UML 2 glasklar: Praxiswissen für die UML-Modellierung, 3. Auflage. Hanser, 2007.
- OMG Object Management Group:
Unified Modeling Language: Infrastructure / Superstructure (2.4.1). OMG, 2011.
- OMG Object Management Group:
Unified Modeling Language (2.5). OMG, 2013.



Software-Architekturen

—

Software-Engineering

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

**Die Technologie- und Management-Disziplin für
die systematische Erstellung und Wartung von
Software.**

+HKA Was ist Software-Engineering?

„Software-Engineering zielt auf die ingenieurmäßige
Entwicklung, Wartung, Anpassung und Weiterentwicklung
großer Software-Systeme
unter Verwendung bewährter, systematischer Vorgehensweisen,
Prinzipien, Methoden und Werkzeuge hin.“

[M. Broy und D. Rombach.

Software Engineering: Wurzeln, Stand und Perspektiven.

Informatik Spektrum 25(6), Springer-Verlag, Heidelberg, 2002.]

Zur Beherrschung dieser Aufgaben ist ein systematisches Vorgehen unabdingbar.

“Engineers, in general, design and build artifacts based on theories and methods, while working under financial and organizational constraints.”

[Ian Sommerville]

- Systematik im Prozess
- Systematik in der Analyse
- Systematik im Design
- Systematik im Testen
- ...

**Verantwortungsbewusstsein
gegenüber
Auftraggeber, Nutzer, Team, ...!**

Ein Entwicklungsprozess definiert „wer tut was, wann und wie“, um das vorgegebene Ziel zu erreichen. Ein effektiver Prozess stellt Richtlinien bereit, die eine effiziente Entwicklung ermöglichen. Er reduziert die Risiken und erhöht die Vorhersehbarkeit.

- **Technologie:**

Der Prozess muss auf der Technologie beruhen, die heute verfügbar ist. (Sprachen, Betriebssysteme, Entwicklungsumgebungen, Netzwerke, ...)

- **Werkzeuge:**

Der Prozess ist werkzeugunterstützt. Werkzeuge und Prozesse entstehen parallel. Der Prozess unterstützt die Entwicklung des Werkzeugs und umgekehrt.

**Allen modernen Prozessmodellen
gemeinsam ist ein agiles, iteratives und
risikoorientiertes Vorgehen.**

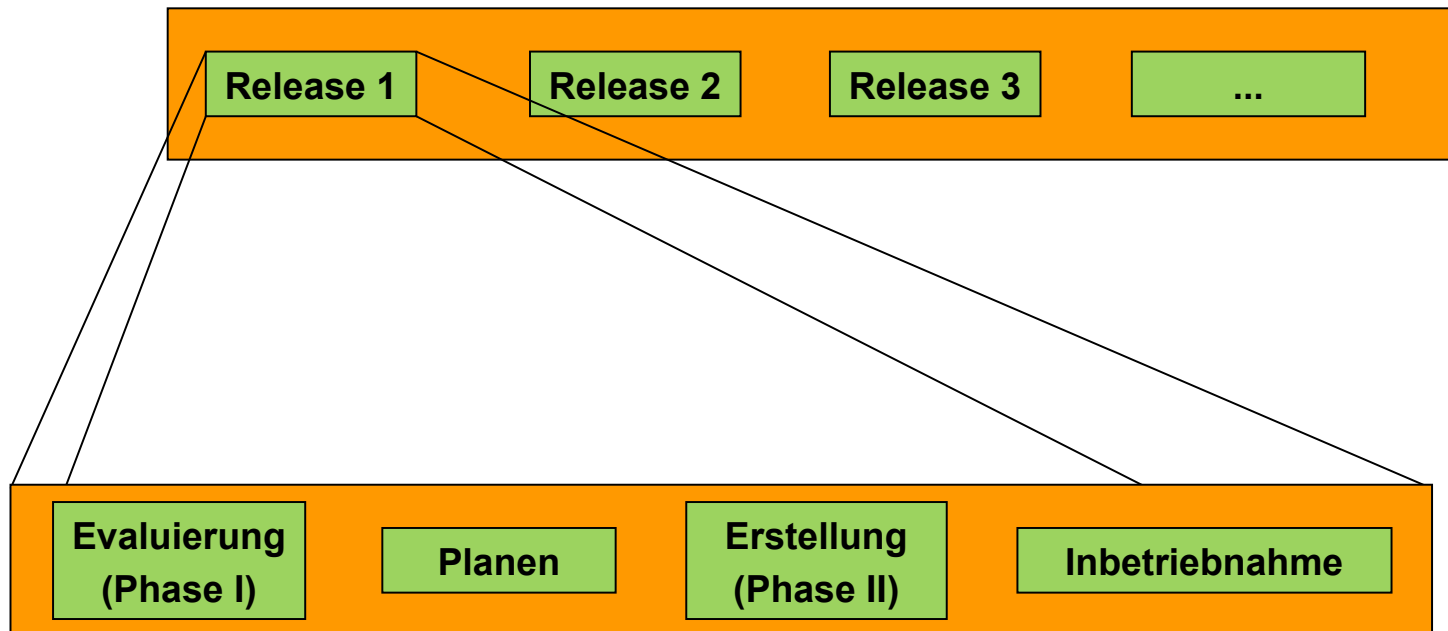
+IKA Vorteile eines iterativen, agilen Vorgehens

- Neu auftretende Nutzeranforderungen können integriert werden.
(Erste Erfahrungen mit lauffähigen Zwischenprodukten.)
- Anforderungen können im Laufe der Versionen verfeinert werden.
- Der frühe Einsatz eines Kernsystems beim Auftraggeber ist möglich.
- Der Auftraggeber kann in die Entscheidung über die Versionsreihenfolge einbezogen werden.
(Wichtige und kritische Aspekte müssen zuerst realisiert werden.)

Erst ein iteratives Vorgehen macht eine strukturierte, agile Software-Entwicklung möglich.

[Larman, Craig: Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 2. ed. Prentice Hall, 2002.]

In vier Schritten zum Release



CI/CD (Continuous Integration and Delivery/Deployment)

- **Continuous Integration (CI):**

Man arbeitet an einer gemeinsamen Code-Basis und verfügt über einen eigenen Feature-Branche. Änderungen werden in regelmäßig (kurzen) Abständen per Pull-Request in die Code-Basis integriert.

- Automatisches Bauen
- Automatisches Testen
- Abgesichert durch ein Quality Gate

- **Pull-Request:**

Der Pull-Request entspricht einer Inversion of Control. Änderungen am Main-Branch kann man nicht selbst durchführen, sondern sie müssen von diesem gezogen werden. Hierdurch kann sichergestellt werden, dass nur geprüfter Code integriert wird..

CI/CD (Continuous Integration and Delivery/Deployment)

- **Continuous Delivery (CD):**
Die Software ist jederzeit auslieferbar. Die Bereitstellung (Deployment) erfolgt aber weiterhin nicht automatisch und ist im Allgemeinen verbunden mit einer manuellen Freigabe der Software.
- **Continuous Deployment (CD):**
Nach einem erfolgreichen Build erfolgt ein automatisiertes Deployment. Eine manuelle Freigabe entfällt.

+IKA Continuous Deployment – Zielplattformen

- **Container- und Orchestrierungsplattformen (CaaS):**

Man stellt Container-Images bereit und nutzt Orchestrierungsdienste, um diese zu starten, zu stoppen, usw.

Kubernetes, Amazon Elastic Container Service (ECS)

- **Cloud-Plattformen (PaaS):**

Man nutzt eine definierte Plattform, die den Betrieb der Anwendung übernimmt und regelt.

Azure, Google Cloud Platform, ...

- **Hosting-Plattformen (SaaS):**

Man nutzt vorgefertigte Plattformen und stellt lediglich den Code der Anwendung bereit. Die Verwaltung der Server entfällt.

Netlify, Vercel, ...

+IKA Continuous Deployment – Zielplattformen

- **Server und Virtuelle Maschinen (IaaS):**

Man nutzt lediglich vordefinierte Zielsysteme, die typischerweise in der Form virtueller Maschinen (Linux, Windows, ...) bereitgestellt werden.

- VMware:
- Amazon Elastic Compute Cloud (EC2)

- **App Stores und Distributionsplattformen:**

Man nutzt vorgefertigte und etablierte Distributionswege.

**Der CI-Prozess sichert die Code-Qualität.
Der CD-Prozess garantiert die Auslieferung bzw.
Bereitstellung des korrekten Codes.**

+HKA Typischer CI/CD-Cycle

1) Pull-Request:

Code-Änderungen erfolgen im Feature-Branch und werden per Pull-Request eingereicht.

2) Die CI-Pipeline startet:

Checkout → Build → Unit- und Integrationstest sowie Code-Analyse.

3) Quality Gate:

Die Ergebnisse der Tests und Analysen werden bewertet.

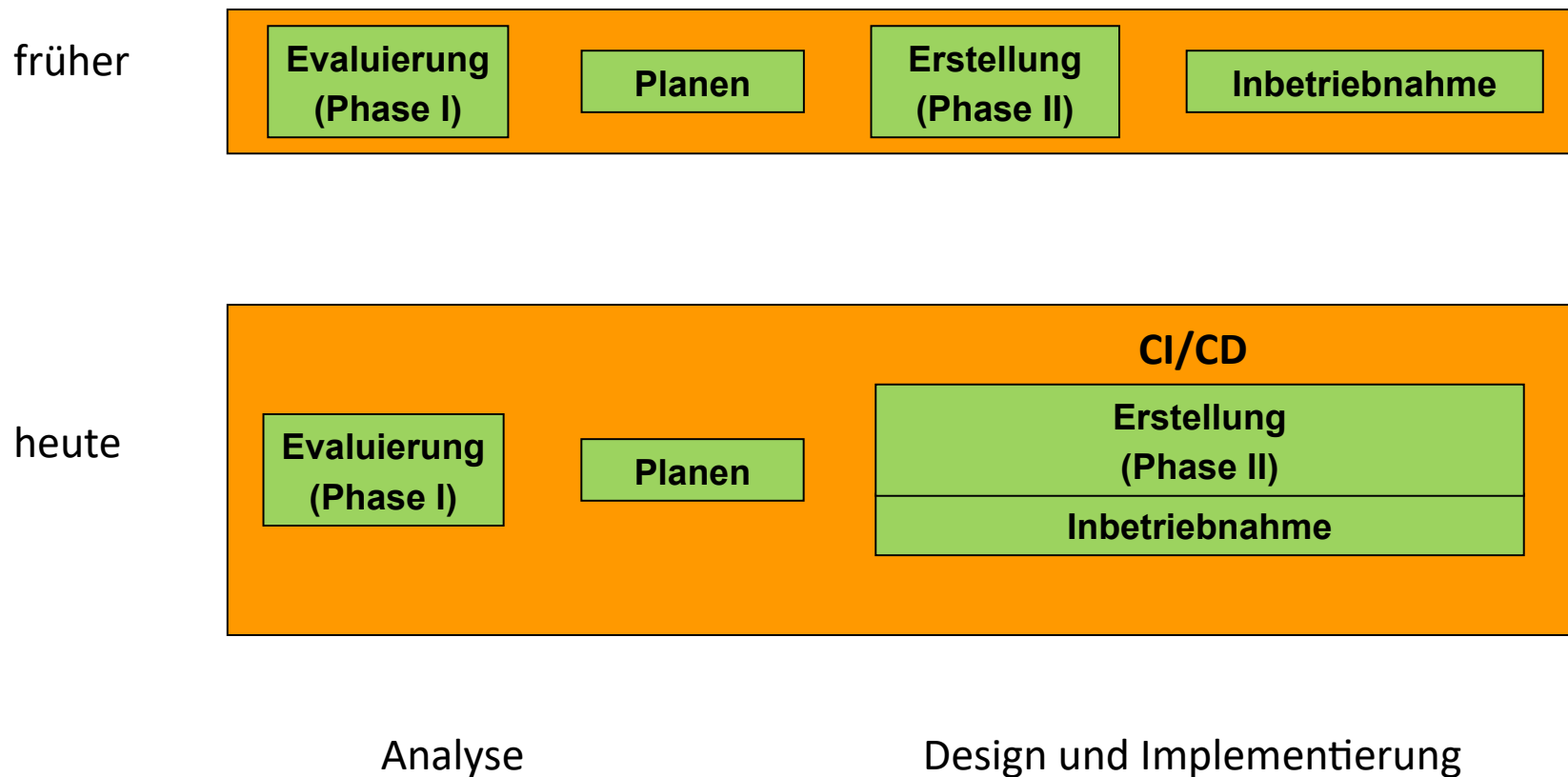
4) Merge und Container Build:

Nach bestandener Kontrolle wird der Pull-Request genehmigt und der Code in den Main-Branch gemergt. Das Image wird gebaut und versioniert.

5) Continuous Delivery / Deployment

Das Deployment-Manifest wird aktualisiert und referenziert das neue Image. Das Image wird in eine Registry gepusht und die Zielplattform z.B. Kubernetes zieht es dann selbstständig.

Und dennoch bleiben die zentralen Herausforderungen gleich!



+IKA Der grobe Ablauf

- **Evaluierung:**
Anforderungen (Requirements) definieren, Prototypen erstellen und erste Iterationen durchführen, bis man die Komplexität der Aufgabe einschätzen kann.
Ziel: Probleme verstehen, Planungssicherheit schaffen.
Primäre Architekturentscheidungen treffen!
- **Planen:**
Zeit, Budget, Personal, usw. anpassen, Backlogs füllen.
- **Erstellung und Inbetriebnahme (CI/CD):**
Das eigentliche Produkt realisieren, neue Anforderungen integrieren, Fehler beseitigen, ...

+IKA Die Herausforderungen

- **Frühe Phasen:**

Architektur konsolidieren!

In den ersten Iterationen der eigentlichen Entwicklung sollten die zentralen, domänenspezifischen Use Cases umgesetzt werden, um eventuell falsche Architekturentscheidungen zu identifizieren und zu korrigieren.

- **Späte Phasen:**

Architektur bleibt stabil!

Die späteren Iterationen konzentrieren sich auf die Vervollständigung der Funktionalität. Dies sollte die Architektur nur unwesentlich beeinflussen.

- Schnittstellen werden erweitert.
- Neue Komponenten kommen hinzu.

Analyse heißt:

Das Problem verstehen und beschreiben, sodass ein korrekter Entwurf möglich wird.
Hierzu wird ein Analysemodell erstellt.

- Use-Case-Diagramme und Activity-Diagramme
- Klassendiagramme und (System-)Sequenz-Diagramme
- ...

Beschrieben werden:

- Die zentralen Anforderungen
- Die zentralen Abläufe
- Die zentralen Konzepte und Objekte
- Die zentralen Ereignisse und Operationen

Oder kurz:

Welche Schnittstellen hat das System?

- Was sind die zusammenhängenden funktionalen Einheiten?
- Welche Schnittstellen gibt es?
- Welche Operationen werden an den Schnittstellen gebraucht?
- Was ist deren Aufgabe?
- Welche Konzepte sind relevant und wie hängen sie voneinander ab?

Design heißt:

Die Lösung beschreiben, sodass die während der Analyse identifizierten Anforderungen, im Sinne des Kunden (der Stakeholder) umgesetzt werden.

Dies umfasst:

- **Systementwurf (oftmals das Bindeglied zwischen Analyse und Design)**
 - Aufbrechen des Systems in Teilsysteme
 - Strategien für Steuerung und Datenhaltung

Primäre Architekturentscheidungen treffen!

- **Objektentwurf (Design)**
 - Anreichern des Analysemodells
 - Umsetzung der Strategien des Systementwurfs
 - Implementierungsentscheidungen treffen
 - Algorithmen entwerfen

Oder kurz:

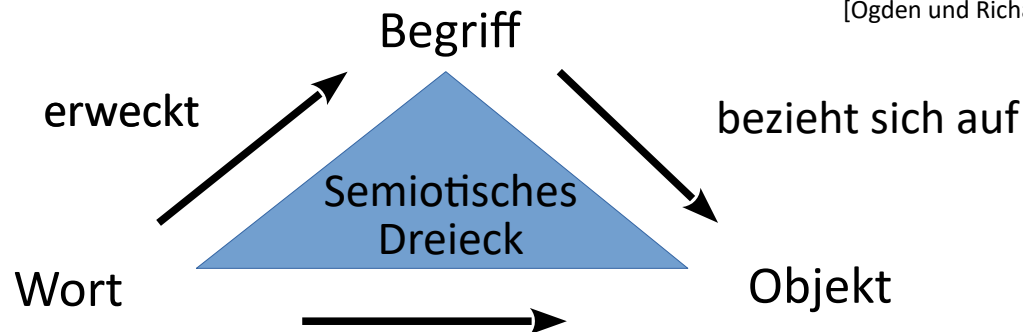
Wie wird die Aufgabe (Schnittstelle) realisiert?

- Architektur: Das System organisieren und die Zugriffsmöglichkeiten definieren.
- Bedienkonzept: Das Erscheinungsbild der Schnittstelle präzisieren.
- **Verantwortlichkeiten zuweisen (der eigentliche Entwurf)**
 - **Verantwortlichkeiten, etwas zu tun (selbst tun oder delegieren)**
 - **Verantwortlichkeiten, etwas zu wissen (speichern, berechnen, assoziieren)**
- Reorganisation der Objekte

Die Operationen der Schnittstelle entwerfen.

+IKA Systematik im Problem

[Ogden und Richards: The Meaning of Meaning, 1923.]



- **Wort:**
Zeichen, Symbole, Modelle, ...

Die Beschreibung von etwas.

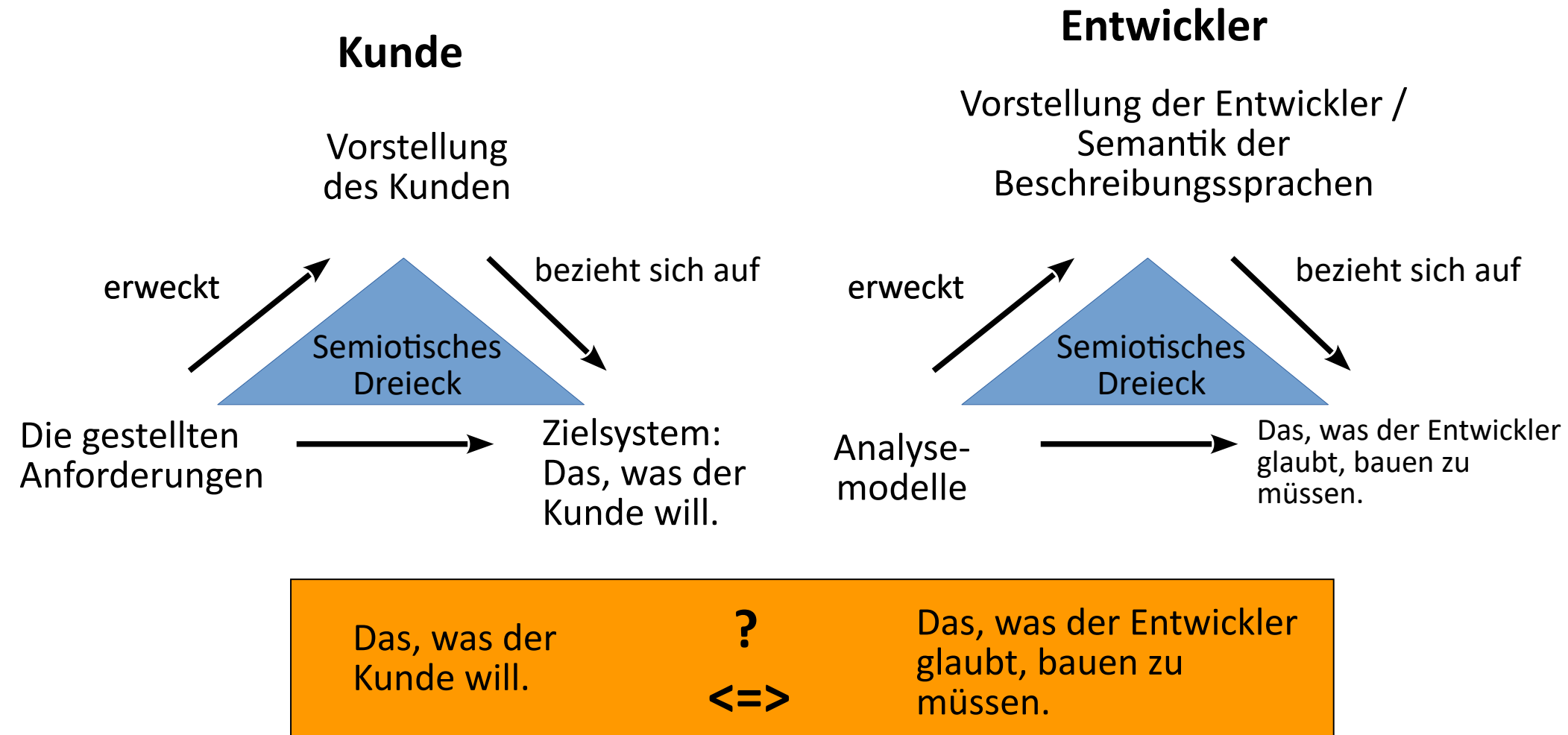
- **Objekt:**
Ding, Realität, ...

Das, was man wirklich will.

- **Begriff:**
Denkeinheit, Semantik, ...

Was steckt dahinter?

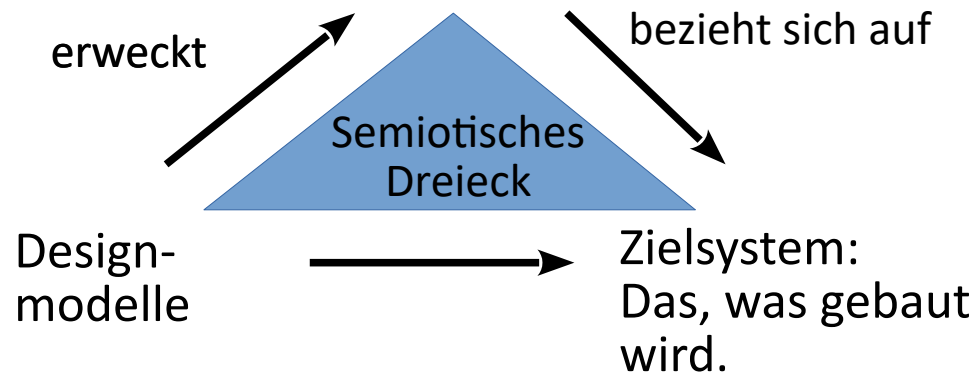
+IKA Systematik im Problem (Analyse)



+HKA Systematik im Problem (Design)

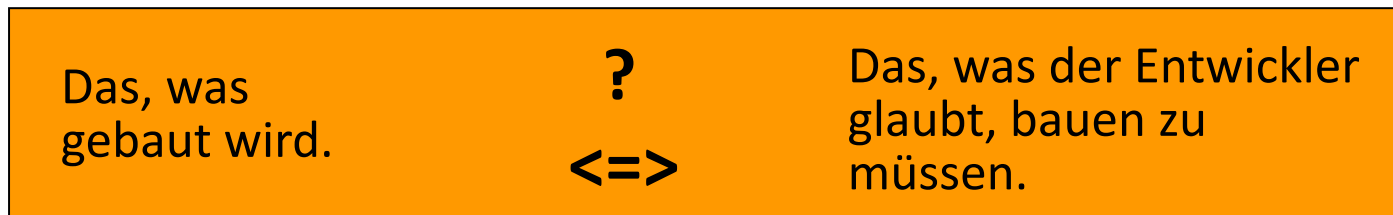
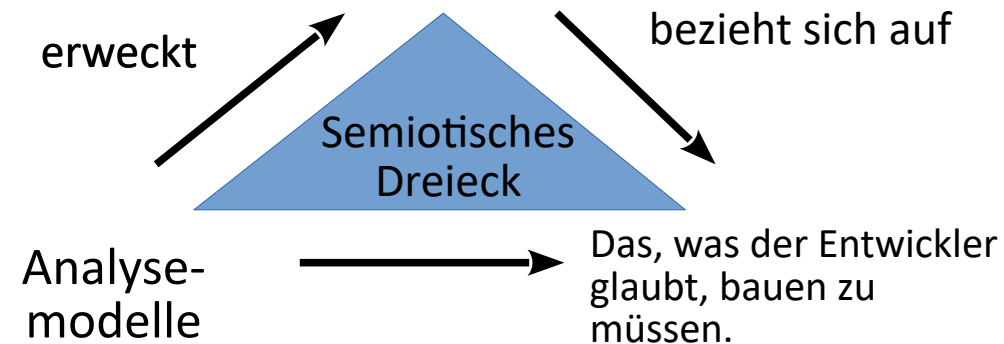
Entwickler

Vorstellung der Entwickler /
Semantik der
Beschreibungssprachen



Entwickler

Vorstellung der Entwickler /
Semantik der
Beschreibungssprachen



+IKA Systematik im Problem (Lösung)

